



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

EXTENSIÓN EXPERIMENTAL DE APPLICATIVO EN GHC MEDIANTE
REORDENAMIENTO DE MÓNADAS CONMUTATIVAS EN PROGRAMAS
PROBABILÍSTICOS DE HASKELL

PROPUESTA DE TEMA DE MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

ESTEBAN ROMERO BERRÍOS

MODALIDAD:
MEMORIA

PROFESOR GUÍA:
FEDERICO OLMEDO

PROFESOR COGUÍA:
ISMAEL FIGUEROA

SANTIAGO DE CHILE
2026

1. Introducción

Los programas probabilísticos constituyen una herramienta de suma relevancia en la actualidad, debido a que estos permiten razonar sobre distribuciones de probabilidad de manera directa; aquello entrega la posibilidad de modelar fenómenos inciertos y predecir eventos en el mundo real.

En particular, estos programas han demostrado ser fundamentales para diversas áreas de la computación, así como en ciberseguridad, mediante sus algoritmos de generación de números aleatorios, al igual que la criptografía, mediante sus protocolos de seguridad. Asimismo, en campos más recientes, este paradigma resulta clave para el entrenamiento de distintos modelos de Machine Learning y el desarrollo de sistemas de Inteligencia Artificial.

Cabe destacar que, en la mayoría de los lenguajes de programación tradicionales, la ejecución de un programa probabilístico tiende a reducirse a la obtención de una sola muestra. Un ejemplo de esto sería que, si se declara la abstracción de un lanzamiento de moneda, el cómputo retornaría únicamente un resultado, el cual sería “cara” o “sello”; por tanto, esta restricción limita la expresividad y la versatilidad del programa. En cambio, un lenguaje puro y funcional como lo es **Haskell**, implementa programas probabilísticos mediante el uso de mónadas de probabilidad, las cuales nos permiten generar y computar todos los posibles resultados de una distribución de probabilidad implícitamente definida por un programa, considerando de forma simultánea cada evento de esta misma; este proceso es conocido como *inferencia*.

Gracias a la forma en la cual se implementan las mónadas de probabilidad en **Haskell**, es posible realizar consultas y operaciones de manera directa sobre la distribución de salida de un programa. Por ejemplo, si se desea inferir la distribución asociada al lanzamiento de una moneda seguido del lanzamiento de un dado, la *do-notation* de las mónadas de **Haskell** permite usar la siguiente sintaxis:

```
flipCoinRollDice = do
  c <- uniform [Head, Tail]
  d <- uniform [1 .. 6]
  return (c, d)
```

En el código anterior, se logra apreciar que la variable `c` codifica cada evento de la primera distribución de probabilidad (moneda) y, por otro lado, la variable `d` hace lo propio, de manera análoga, con la segunda (dado); al formar la tupla con ambos eventos, se obtiene la distribución correspondiente a la ejecución conjunta de los dos experimentos. Siendo más precisos, esta nueva distribución estaría compuesta por doce tuplas de eventos.

Es pertinente recalcar que, en el programa anterior, las dos variables que representan cada distribución son independientes entre sí; en otras palabras, se podría conmutar el orden de sus declaraciones y el resultado sería el mismo. Esto propone la posibilidad de aprovechar los recursos disponibles y ejecutar en paralelo el cálculo de cada distribución por separado. Si bien, en el ejemplo mostrado, la paralelización puede parecer innecesaria, en escenarios reales con millones de posibles eventos por distribución resulta fundamental, ya que se deben generar todas las posibles opciones. Por consiguiente, el proceso de inferir nuevos resultados dadas

unas distribuciones iniciales (moneda y dado en el ejemplo) tiene un costo computacional elevado.

Al igual, es importante señalar que los programas probabilísticos en `Haskell` generalmente presentan una complejidad superior al ejemplo básico anteriormente mencionado. Es común que existan dependencias entre las declaraciones de cada distribución, lo cual se muestra en el siguiente ejemplo:

```
do
  d1 <- uniform [1 .. 6]
  d2 <- uniform [1 .. 6]
  d3 <- uniform [1 .. d1]
  d4 <- uniform [1 .. d2]
  return (d3, d4)
```

Esto modela la inferencia del lanzamiento de dos dados de forma simultánea `d3` y `d4`, con la particularidad de que la cantidad de caras de cada dado está determinada por el resultado obtenido en los primeros lanzamientos de `d1` y `d2`. Con esto, en la instrucción `return` se generará una nueva distribución con todas las tuplas posibles al lanzar los dados `d3` y `d4` después del proceso de selección de caras definido por `d1` y `d2`. Por lo que las probabilidades resultantes difieren de las habituales.

En este caso, para obtener la tupla $(6,6)$ en la distribución de salida, es necesario que en ambos lanzamientos de `d1` y `d2` se obtenga el número 6. Luego, se determina que los dados `d3` y `d4` tendrán 6 caras cada uno y, finalmente, deberán arrojar el número 6 cada uno por separado. En consecuencia, la tupla $(6,6)$ tiene una probabilidad asociada de $1/1296$.

El problema yace en que, el uso de la *do-notation* para representar programas probabilísticos no permite la paralelización de los mismos. Esto se debe a que codifica cómputos inherentemente secuenciales; en el primer ejemplo mostrado, la variable `c` del primer muestreo queda en el contexto del resto del cuerpo del *do-return*; es decir, puede ser usada para la propia declaración de la variable `d`. Esto mismo sucede en el segundo ejemplo; el resultado del lanzamiento de los primeros dados se consume en la declaración de los dados `d3` y `d4`.

Una alternativa para resolver esto sería escribir el programa con otra sintaxis, pero manteniendo la misma semántica, de modo que la ejecución pueda ser paralelizada. Sin embargo, esto conllevaría la pérdida de la *do-notation*, la cual se usa de forma universal en `Haskell`; lo ideal sería que esta pudiera seguir utilizándose.

Finalmente, el hecho de poder paralelizar el cómputo de las mónadas de probabilidad, lograría una optimización del tiempo que impactaría directamente en todas las áreas que hacen uso de estas estructuras; por lo cual, resultaría especialmente valioso encontrar un método que permita paralelizar las mónadas de probabilidad sin sacrificar su notación versátil y universal. En el ejemplo, debería identificarse, por un lado, el subcómputo asociado a `d1` y `d3`. Mientras que, por otro lado, el subcómputo asociado a `d2` y `d4`.

El conflicto abordado en esta memoria se sitúa en la etapa inicial de la paralelización de programas probabilísticos en **Haskell**. Más específicamente, se centra en extender el algoritmo actualmente utilizado por **GHC**¹ para la transformación de expresiones monádicas en expresiones paralelizables en el proceso de compilación. De modo que este pueda considerar reordenamientos semánticamente válidos de *statements* dentro de la *do-notation* cuando se trabaja con mónadas conmutativas o su ejemplo canónico, monadas de probabilidad.

El objetivo de la memoria es mejorar el plan de ejecución generado por el compilador de **Haskell**, ampliando el espacio de configuraciones que el algoritmo puede evaluar sin alterar la semántica del programa. En particular, este trabajo basará su solución en los procedimientos y algoritmos propuestos por Marlow et al. [7] que ya se encuentran implementados en **GHC**.

2. Preliminares

En primer lugar, es necesario fijar las nociones de *functor* y *aplicativo* en **Haskell**, ya que las *mónadas* se introducen en el lenguaje como una extensión de dichas abstracciones. En particular, la noción de mónada fue presentada por Wadler en 1995 [11]. Por consiguiente, se comenzará por los *functores*; posteriormente, se abordará su respectiva subclase, los *aplicativos*, presentados por McBride y Paterson en 2008 [8]; y, finalmente, las *mónadas*, que constituyen una subclase de los aplicativos.

Una vez establecida esta base, se introducirá el caso particular de las *mónadas de probabilidad*, ya que constituyen el dominio principal de aplicación de esta memoria y permiten conectar la teoría monádica con programas probabilísticos concretos en **Haskell**.

Al igual, es importante mencionar que, al ser **Haskell** un lenguaje funcional, las funciones se tratan como valores de primera clase, lo que permite, entre otras cosas, la *curriificación* y *aplicación parcial* de estas mismas. Un ejemplo claro es el operador (+), cuyo tipo puede representarse como $(+) :: \text{Num} \rightarrow \text{Num} \rightarrow \text{Num}$. Si aplicamos parcialmente (+) al valor 5, obtenemos la función unaria (+5), de tipo $\text{Num} \rightarrow \text{Num}$. En otras palabras, al aplicar la función con un solo argumento, se retorna una nueva función que espera el argumento restante.

2.1. Functores

En **Haskell**, un *functor* es una `typeclass` que representa a toda estructura que permita aplicar una función a los valores que contiene, sin la necesidad de cambiar la estructura en sí [12]; por esto, se puede imaginar un functor como un contenedor. A partir de esta noción, para ejecutar cálculos sobre los valores contenidos en él, se implementa la función canónica de los functores, `fmap`. Esta función permite *desempaquetar* un functor, aplicar una función `f` sobre su contenido y luego *empaquetar* nuevamente el resultado. Además, `fmap` debe respetar las leyes de identidad y composición, lo cual garantiza que el mapeo preserve la estructura abstracta del contenedor [12].

¹ *Glasgow Haskell Compiler* es el compilador estándar, de código abierto y más utilizado para el lenguaje de programación funcional **Haskell**

```
class Functor F where
  fmap :: (a -> b) -> F a -> F b
```

Los ejemplos clásicos de funtores son las listas y árboles, debido a que, al aplicar `fmap` se transforman sus datos de manera uniforme, manteniendo la estructura subyacente.

2.2. Aplicativos

La `typeclass applicative` de Haskell, caracteriza la abstracción de un estilo aplicativo de programación, más débil que las mónadas y, por tanto, más ampliamente utilizado [8]. Por ende, la necesidad de aplicar funciones más complejas a los funtores; como lo sería una función binaria, se ve resuelta. Por ejemplo, para darle un caso de aplicación, tomaremos la función `(+)`.

Para resolver y aplicar la función antes mencionada, un functor aplicativo, permite operar con valores que ya se encuentran dentro de un contenedor a otro contenedor. Por lo cual, consideremos el aplicativo `Maybe Int`, que posee dos constructores: `Just Int` y `Nothing`. En el caso de sumar `Just 2` y `Just 3`, primero aplicamos `(+)` al primer contenedor, desempaqueando el número 2 y empaquetando la función ahora unaria `(+2)` en un nuevo aplicativo `Just (+2)` siendo de tipo `Maybe (Int -> Int)`. Luego, se aplica el contenido de este último al segundo contenedor, desempaqueando el número 3, realizando la operación y empaquetando el resultado 5 en un nuevo aplicativo `Just 5`.

Lo más destacable de los aplicativos es que permiten manejar cada abstracción específica dentro de sus propias implementaciones. Por ejemplo, para la clase `Maybe Int`, si alguno de los dos sumandos es `Nothing`, se produce un cortocircuito que retorna al instante `Nothing`; cuya implementación se mostrará en los siguientes párrafos. De esta forma, `Maybe Int` actúa como un aplicativo que evita la necesidad de implementar sumas para cada aridad con múltiples líneas de *Pattern Matching* anidado para verificar si un sumando es `Just Int` o `Nothing`.

De este modo, los funtores aplicativos permiten aplicar funciones con un número arbitrario de argumentos sin dificultad. Estos mismos, se describen mediante la siguiente definición:

```
class Functor F => Applicative F where
  pure :: a -> F a
  <*> :: F (a -> b) -> F a -> F b
```

En particular, el *aplicativo* `Maybe T` considerando un tipo genérico `T`, implementa los operadores `pure` y `<*>` conforme a su abstracción funcional específica, es decir, trabajar con valores opcionales dentro de un programa, y combinar operaciones que pueden fallar. Dado lo anterior, se muestran sus respectivas definiciones en el siguiente fragmento de código:

```
instance Applicative Maybe where
  pure = Just
  Nothing <*> _ = Nothing
  _ <*> Nothing = Nothing
  (Just f) <*> (Just x) = Just (f x)
```

Nótese que, en caso de que operar con algún objeto no definido, cuya abstracción equivale al constructor `Nothing`, las siguientes aplicaciones retornarán a su vez un elemento `Nothing`, modelando así el comportamiento de la clase `Maybe T`. En caso contrario, si ambos operandos se encuentran definidos, es decir, son elementos encapsulados en el constructor `Just`, se procede con la aplicación del primero sobre el segundo, para finalmente empaquetar el elemento resultante dentro del mismo constructor `Just`.

Volviendo al ejemplo de aplicación de la suma (+) sobre elementos de tipo `Maybe Int`, se requiere que esta se encuentre encapsulada dentro del constructor `Just`, ya que, si no lo estuviera, se rompería el invariante de desempaquetado y empaquetado asociado a la implementación del operador `<*>`. Para resolver esto, es preciso utilizar `pure`, que permite empaquetar la función `f` desde un principio. De este modo, la instancia concreta de la suma sobre `Maybe Int` tendría la siguiente sintaxis:

```
pure (+) <*> Just 2 <*> Just 3
```

Posteriormente, para que la notación sea más limpia y versátil, se introduce el operador `<$>`, el cual satisface la siguiente igualdad: `f <$> m = pure f <*> m`. Esto quiere decir que, dada una función no empaquetada `f` y un aplicativo `m`, se empaqueta la respectiva función con `pure` y luego se aplica. Por tanto, el ejemplo anterior se vería modificado, simplificando su sintaxis:

```
(+) <$> Just 2 <*> Just 3
```

Finalmente, nótese que en una expresión aplicativa, la estructura del cómputo queda fijada de antemano: cada argumento se evalúa en su propio contexto y luego se combinan sus resultados mediante la aplicación de una función pura. Esta característica hace que, a nivel semántico, los subcómputos asociados a los argumentos sean independientes entre sí (en el sentido de que ninguno puede *usar* el resultado del otro para definirse), lo cual habilita optimizaciones como evaluación paralela cuando el contexto lo permite [8].

Sin embargo, esta misma restricción implica que no es directo expresar dependencias entre argumentos. En particular, si se quisiera que el segundo argumento dependa del valor producido por el primero, el estilo aplicativo resulta insuficiente, ya que la forma del cómputo no puede adaptarse dinámicamente a resultados intermedios. Asimismo, existen situaciones en que la función a aplicar es de tipo `a -> F b`; es decir, retorna un valor ya encapsulado en el contexto, lo que rompe la firma esperada requiriendo un mecanismo adicional, lo cual motiva la introducción de mónadas [11].

2.3. Mónadas

Las *mónadas* de `Haskell` son una `typeclass` que describe cómo encadenar operaciones que producen o manejan efectos, manteniendo un flujo de ejecución ordenado dentro de un contexto [11]. Esto nos ayuda a solucionar los dos problemas descritos anteriormente mediante el operador `bind`, el cual es escrito como `M a >>= f`. Este operador desempaqueta la mónada de la izquierda y aplica la función de la derecha sobre el elemento `a`, de manera que lo retornado por la ejecución de `f a` sea nuevamente una mónada `M b`.

```
class Applicative M => Monad M where
  return :: a -> M a
  (>=)   :: M a -> (a -> M b) -> M b
```

De esta manera, si queremos concatenar múltiples aplicaciones a distintas mónadas como con el operador `<*>`, hace falta declarar funciones anónimas λ que nos permitan representar un flujo claro del código. En vista de lo expuesto, resulta que el tipo `Maybe T` también es una mónada, y para ilustrar el comportamiento de estas mismas, se utilizará el mismo ejemplo de la suma (+) descrito en la sección anterior.

En este caso, la implementación de los operadores `return` y `>=`, que satisfacen el comportamiento de la clase `Maybe T` con `T` un tipo genérico, es la siguiente:

```
instance Monad Maybe where
  return x = Just x
  Nothing >= _ = Nothing
  (Just x) >= f = f x
```

Por consiguiente, al igual que los aplicativos, la mónada `Maybe T` implementa el operador `bind` tal que respete la abstracción asociada a esta clase, es decir, trabajar con elementos opcionales. Por ende, en caso de aplicar una función a un elemento no definido, dicho de otro modo, que utiliza el constructor `Nothing`, se retornará este mismo valor. En contraste con lo anterior, si el elemento a desempaquetar es de tipo `Just`, se ejecuta la función `f` sobre este. En este caso, la función que se ejecuta debe respetar la firma de tipos y retornar una nueva mónada. Ya explicado esto, el código equivalente a la aplicación de la función (+) pero usando notación monádica es el siguiente:

```
Just 3 >= (\x -> Just 2 >= (\y -> return ((+) x y)))
```

Por tanto, al ejecutar el operador `>=`, se desempaqueta el número 3 y se aplica la función anónima de la derecha sobre este valor, reemplazando todas las apariciones del identificador `x` por 3. A continuación, se realiza el mismo proceso con la mónada que contiene el 2, resultando finalmente en la instrucción `((+) 3 2)` que será empaquetada gracias a la instrucción `return` retornando la mónada `Just 5`.

Cabe recalcar que ahora el identificador `x` está disponible en todo el *scope* de la primera función anónima. Por lo que, `Just 2` podría perfectamente usar `x` para su definición, por ejemplo `Just ((* ) 2 x)`. Esto nos permite declarar dependencias entre argumentos, lo que a su vez obliga a la ejecución secuencial de las funciones anónimas.

En consecuencia, al reordenar el código de mejor manera quedaría:

```
Just 3 >= (\x ->
  Just 2 >= (\y ->
    return ((+) x y)))
```

Luego, *Haskell* agrega *azúcar sintáctica*² a la aplicación del operador `>>=`, de esta manera se presenta la *do-notation*:

```
do
  x <- Just 3
  y <- Just 2
  return ((+) x y)
```

Por último, existe el operador `join`, que se encarga de aplanar mónadas anidadas de la forma `m (m a)` y retorna un valor de tipo `m a`. Un ejemplo muy simple usando la mónada `Maybe Int` sería el siguiente:

```
join (Just (Just 1)) = Just 1
```

2.4. Mónadas de probabilidad

En el marco de la programación probabilística, una *mónada de probabilidad* permite interpretar un programa como un proceso generativo que induce una distribución sobre sus posibles resultados. Desde esta perspectiva, las primitivas probabilísticas (por ejemplo, “muestrear” desde una distribución base) se integran en una interfaz monádica estándar: `return` representa una distribución degenerada sin incertidumbre, mientras que el operador `bind >>=` permite encadenar elecciones aleatorias, de modo que decisiones posteriores puedan depender de resultados previos. En consecuencia, la *do-notation* de *Haskell* se vuelve una sintaxis particularmente natural para expresar modelos probabilísticos con dependencias de datos, condicionales y estructura secuencial.

El trabajo de Ścibior muestra que el estilo monádico puede utilizarse de forma práctica en modelos de aprendizaje automático y estadística bayesiana, siempre que se acompañe de mecanismos de inferencia eficientes [10]. En particular, se mantiene una interfaz monádica para construir modelos, mientras la inferencia opera sobre una representación interna de las distribuciones; además, se subraya la utilidad de contar con una semántica formal para precisar equivalencias entre programas.

Para esta memoria, este antecedente es relevante porque (i) confirma que la *do-notation* monádica es un patrón central para especificar modelos probabilísticos en *Haskell* y (ii) evidencia que la secuencialidad sintáctica puede ocultar independencia entre subcómputos. Por ello, una transformación que haga explícita dicha independencia preservando la semántica probabilística, puede habilitar reescrituras más paralelizables y reordenamientos seguros cuando corresponda.

²La *azúcar sintáctica* busca hacer el lenguaje más digerible sin alterar la semántica del programa [9]. Por ende la esencia de la aplicación del operador `bind` no se ve alterada.

3. Estado del Arte

Las mónadas de Haskell utilizan la *do-notation* para facilitar la lectura y escritura de código. Posteriormente, al compilar el programa, el compilador extrae la azúcar sintáctica y reemplaza la declaración de las expresiones monádicas por la evaluación de funciones λ , estas se ejecutarán de forma secuencial debido a la propia definición del operador *bind*, asociada a cada mónada específica.

Dado lo anterior, implementar la paralelización de las expresiones monádicas dentro de la *do-notation* no es algo trivial de resolver. Como consecuencia del interés general de solucionar el problema anterior, este ya ha sido abordado anteriormente, por Marlow, Jones, Kmett y Mokhov en 2016, cuya solución propuesta consiste en transformar el código fuente secuencial de una mónada en una representación intermedia paralelizable, mediante el uso de funtores aplicativos. De esta manera, se modifica la sintaxis y se mantiene la semántica del programa [7]. En dicho trabajo, se plantea la transformación para una mónada arbitraria; es decir, esta puede contener dependencias entre declaraciones de argumentos.

3.1. Pipeline de transformación de *do-notation*: Rearrangement y Desugaring

El algoritmo presentado por Marlow, Jones, Kmett y Mokhov en 2016 consiste en un *pipeline* de dos etapas, *rearrangement* y *desugaring* [7, p. 96]. Cada etapa comprende múltiples pasos; sin embargo, nos centraremos principalmente en la fase de *rearrangement*, ya que en esta se genera el plan de ejecución óptimo para la paralelización de las expresiones monádicas. Estas últimas serán denominadas *statements* de la *do-notation* de aquí en adelante.

En términos de implementación y reproducibilidad, resulta especialmente relevante que esta transformación no solo exista como propuesta académica, sino que se encuentre incorporada y documentada en el compilador *Glasgow Haskell Compiler* (GHC) como la extensión `ApplicativeDo` [6]. En particular, la documentación oficial de GHC describe explícitamente el propósito de la extensión: reescribir bloques `do` utilizando operadores aplicativos (`<$>`, `<*>`) y, cuando corresponde, el operador `join`, de modo de maximizar el uso de una estructura de efectos estática “tanto como sea posible” sin alterar la semántica del programa [7].

Esta especificación es importante para el estado del arte por dos razones: primero, porque fija un contrato público (respaldado por el compilador) respecto del comportamiento esperado de la transformación (qué formas de `do` pueden “desazucararse” a aplicaciones paralelizables y bajo qué limitaciones); y segundo, porque permite que cualquier propuesta de mejora (como la que plantea esta memoria) se formule en continuidad con una base establecida y validada por la comunidad, con un punto de comparación claro tanto a nivel de semántica como de comportamiento del compilador. En consecuencia, el hecho de que `ApplicativeDo` esté normada en el manual de GHC transforma el problema desde una idea puramente conceptual a una línea de trabajo verificable sobre una herramienta real, con pruebas y casos de uso existentes, lo que refuerza la factibilidad técnica de extender el pipeline descrito por Marlow et al [7].

3.1.1. Etapa de Rearrangement

En primer lugar, sea una expresión monádica arbitraria, la cual usa la *do-notation*. En ella, se declaran múltiples *statements* de la forma **pi** <- **ei**, estos últimos se denotarán **si**, donde **i** es el índice asociado a cada uno de ellos. Posteriormente, este fragmento de código se transformará en un elemento de la sintaxis abstracta **do** **l** **e**, donde **l** es la lista de *statements* **{s1; ...; sn}**.

Una vez que la expresión **do** ha sido transformada en un elemento de la sintaxis abstracta, el conjunto de *statements* **l** se introduce como entrada al proceso de *rearrangement*, el cual no considera la expresión **e** dentro de sus cálculos. Este proceso tiene la siguiente descripción, desarrollada por Marlow y los otros autores antes mencionados:

$$\begin{aligned}
\text{rearrange } \{s_1; \dots; s_n\} &= \{s_1\}, & \text{if } n = 1 \\
&= \text{split } g_1, & \text{if } k = 1 \\
&= \{(\text{split } g_1 \mid \dots \mid \text{split } g_k)\}, & \text{otherwise} \\
\text{where} \\
g_1 \dots g_k &= \text{segments } \{s_1; \dots; s_n\} \\
\text{segments } \{s_1; \dots; s_n\} &= \{s_1; \dots; s_{i_1}\} \dots \{s_{(i_k)+1}; \dots; s_n\} \\
\text{where} \\
i_1 \dots i_k &= \{ i \in 1 \dots n \mid \text{bv}\{s_1; \dots; s_i\} \cap \text{fv}\{s_{i+1}; \dots; s_n\} = \emptyset \} \\
\text{split}\{s_1; \dots; s_n\} &= \{s_1\}, & \text{if } n = 1 \\
&= \text{splitat } i_{\text{opt}}, & \text{otherwise} \\
\text{where} \\
\text{splitat } i &= \text{rearrange } \{s_1; \dots; s_i\} \mathbin{++} \text{rearrange } \{s_{i+1}; \dots; s_n\} \\
i_{\text{opt}} &\in 1 \dots n \text{ such that} \\
&\forall j. 1 \leq j < n. \text{cost}_s(\text{splitat } j) \geq \text{cost}_s(\text{splitat } i_{\text{opt}}) \\
\text{cost}_s \{s_1; \dots; s_n\} &= \Sigma \{ \text{cost}_a s_i \mid 1 \leq i \leq n \} \\
\text{cost}_a (\mathbf{p} \leftarrow \mathbf{e}) &= 1 \\
\text{cost}_a (l_1 \mid \dots \mid l_n) &= \max \{ \text{cost}_s l_i \mid 1 \leq i \leq n \} \\
\text{fv } \{s_1; \dots; s_n\} &= \text{the free variables of } s_1 \dots s_n \\
\text{bv } \{s_1; \dots; s_n\} &= \text{the variables bound by } s_1 \dots s_n
\end{aligned}$$

Figura 1: Proceso de *Rearrangement*

El proceso ya mencionado *rearrangement* se encarga de generar un plan de ejecución óptimo, asociado a la cadena de *statements* **{s1; ...; sn}**, determinando qué segmentos se pueden paralelizar y cuáles no. La función principal de este proceso toma como nombre **rearrange**. Esta última, en primera instancia, realiza un barrido secuencial de **{s1; ...; sn}** para identificar los segmentos paralelizables de manera inmediata, utilizando e invocando la función **segments**.

Continuando con lo anterior, la función **segments** iterará sobre las posibles separaciones entre los *statements*. En otras palabras, dado un conjunto de **n** *statements*, se consideran las **n-1** separaciones posibles. Para cada separación **i**, se verifica si la intersección entre el conjunto de variables definidas en **{s1; ...; si}** y el conjunto de variables libres en **{si+1, ..., sn}** es vacía. De ser así, se puede realizar un corte en **i**, ya que no existen dependencias entre ambos grupos de *statements*. Por el contrario, si la intersección no es vacía, se descarta el corte en **i** debido a las dependencias existentes.

Al verificar si la intersección es vacía en la función `segments`, se hace referencia a la propiedad *def-use*. Esta propiedad establece que, para utilizar una variable x , es necesario que haya sido definida previamente. Por lo tanto, si se realizara un corte en una posición i de la lista de *statements* donde la intersección no es vacía, esto indicaría que la secuencia $\{s(i+1); \dots; s_n\}$ hace *uso* de una variable *definida* en $\{s_1; \dots; s_i\}$. En consecuencia, al realizar el corte se violaría la propiedad *def-use*.

En el siguiente paso se emplea la función `split`, que itera de manera similar sobre las separaciones de los *statements* $\{s_1; \dots; s_n\}$, pero en este caso aplica `rearrange` por separado a cada par de subconjuntos. Una vez obtenidas todas las combinaciones posibles, selecciona la de menor costo.

Dado lo anterior, es necesario describir el modelo de costos utilizado por este algoritmo y la notación para representar ejecuciones paralelas de *statements*. Para estas últimas, dado un conjunto $\{l_1, \dots, l_n\}$ de secuencias de *statements* que pueden ejecutarse en paralelo, se utilizará en la sintaxis abstracta la notación $(l_1 | \dots | l_n)$. En consecuencia, el modelo de costos se define de la siguiente manera:

$$\begin{aligned} \text{cost}(p \leftarrow e) &= 1 \\ \text{cost}(\{s_1; \dots; s_n\}) &= \sum \{ \text{cost}(s_i) \mid i \in \{1, \dots, n\} \} \\ \text{cost}(l_1 | \dots | l_n) &= \max \{ \text{cost}(l_i) \mid i \in \{1, \dots, n\} \} \end{aligned}$$

Se observa que, el costo de una secuencia de *statements* no paralelizables corresponde a la suma de los costos individuales de cada *statement*. Por otra parte, para un conjunto de secuencias que pueden ejecutarse en paralelo, el costo total será el máximo valor entre los costos de cada secuencia. Finalmente, en el caso unitario de un solo *statement*, el costo será igual a 1.

Finalmente, como se mencionó antes, la función `split` selecciona el plan de ejecución óptimo con el costo mínimo, cuya acción resulta computacionalmente cara debido a las múltiples llamadas recursivas. Posteriormente, luego de producir el nuevo orden de ejecución, este se guarda en el elemento `l'` de la sintaxis abstracta, siendo retornado el objeto `do l' e` para su futuro procesamiento en la etapa de *desugaring*.

3.1.2. Etapa de Desugaring

La siguiente etapa corresponde al proceso de *desugaring*, que aplica *pattern matching* recursivo sobre el objeto retornado por la etapa de *rearrangement* `do l' e`. Los casos que abarca esta etapa se describen en la siguiente figura:

Se observa que, la función `desugar` considera cada escenario posible dentro del *pattern matching*. Este último, se ejecuta sobre la lista de *statements* `l'` que fue generada en el proceso de *rearrangement* de forma previa. Continuando con la descripción de esta función, dependiendo del caso, se usarán los operadores asociados a *functores aplicativos* o a su subclase, las *mónadas*.

$$\begin{aligned}
& \text{desugar} :: \text{Stmts} \rightarrow \text{Expr} \rightarrow \text{Expr} \\
& \text{desugar } \{ \} e = \text{pure } e \tag{0} \\
& \text{desugar } \{ p \leftarrow e \} e' \tag{1} \\
& \quad \begin{array}{l} | p == e' = e \\ | \text{otherwise} = (\lambda p \rightarrow e') <\$> e \end{array} \\
& \text{desugar } \{ p \leftarrow e; l \} e' = e >>= \lambda p \rightarrow \text{desugar } l e' \tag{2} \\
& \text{desugar } \{ (l_1 | \dots | l_n) \} e \tag{3} \\
& \quad = (\lambda p_1 \dots p_n \rightarrow e) <\$> e_1 <*> \dots <*> e_n \\
& \quad \textbf{where } (p_i, e_i) = \text{desugar}_{\text{arg}} l_i \text{fv}(e) \\
& \text{desugar } \{ (l_1 | \dots | l_n); s \} e \tag{4} \\
& \quad = \text{join } ((\lambda p_1 \dots p_n \rightarrow e') <\$> e_1 <*> \dots <*> e_n) \\
& \quad \textbf{where} \\
& \quad \quad e' = \text{desugar } s e \\
& \quad \quad (p_i, e_i) = \text{desugar}_{\text{arg}} l_i \text{fv}(e') \\
& \text{desugar}_{\text{arg}} :: \text{Stmts} \rightarrow \text{Set Var} \rightarrow (\text{Pat}, \text{Expr}) \\
& \text{desugar}_{\text{arg}} \{ p \leftarrow e \} vs = (p, e) \\
& \text{desugar}_{\text{arg}} l vs = ((v_1, \dots, v_k), \text{desugar } l (v_1, \dots, v_k)) \\
& \quad \textbf{where} \\
& \quad v_1 \dots v_k = \text{bv}(l) \cap vs
\end{aligned}$$

Figura 2: Proceso de *Desugaring*

3.1.3. Aplicación del algoritmo

Para ilustrar el flujo del algoritmo descrito, se ingresará como entrada el ejemplo mostrado en la sección 1, aquel que modela el lanzamiento de dos dados, cuya cantidad de caras es determinada por dos lanzamientos previos. Por tanto, el código asociado es el siguiente:

```

do
  d1 <- uniform [1 .. 6]
  d2 <- uniform [1 .. 6]
  d3 <- uniform [1 .. d1]
  d4 <- uniform [1 .. d2]
  return (d3, d4)

```

En primer lugar, este fragmento de código debe ser transformado a un elemento de la sintaxis abstracta de la forma `do 1 e`. Este último, tendría la siguiente estructura:

```

do
  {d1 <- uniform [1 .. 6];
  d2 <- uniform [1 .. 6];
  d3 <- uniform [1 .. d1];
  d4 <- uniform [1 .. d2]} (d3, d4)

```

Continuando con el algoritmo, la lista de *statements* debe ser transformada por el proceso de *rearrangement*. Este mismo, ejecuta **segments** sobre 1, identificando tres posibles puntos de corte. Sin embargo, ninguno de ellos resulta viable debido a las dependencias existentes. Por consiguiente, la función retorna 1 sin modificaciones.

En el siguiente fase, se aplica la función `split` sobre 1. En donde los posibles valores de corte son {1, 2, 3}. Para simplificar la notación, se definirán las siguientes equivalencias:

```
s1 = d1 <- uniform [1 .. 6]
s2 = d2 <- uniform [1 .. 6]
s3 = d3 <- uniform [1 .. d1]
s4 = d4 <- uniform [1 .. d2]
```

Dada la notación anterior, las combinaciones posibles que son generadas por la función `split`, y que posteriormente solo se escogerá la óptima con menor costo, son:

```
i = 1 -> {rearrange s1; rearrange {s2; s3; s4}}
i = 2 -> {rearrange {s1; s2}; rearrange {s3; s4}}
i = 3 -> {rearrange {s1; s2; s3}; rearrange s4}
```

Ignoremos las llamadas recursivas a la función `rearrange` y consideremos solo lo que retornan, de esta forma obtenemos los siguientes costos asociados a cada plan de ejecución posible:

```
i = 1 -> {s1; s2; (s3|s4)} -> cost() = 3
i = 2 -> {(s1|s2); (s3|s4)} -> cost() = 2
i = 3 -> {(s1|s2); s3; s4} -> cost() = 3
```

En este punto, es evidente cuál opción debe seleccionar la función `split`: la segunda opción, con un costo igual a 2. Con ello, la llamada principal a `rearrange` finaliza y se retorna el plan de ejecución {(s1|s2); (s3|s4)} en el objeto `do 1'` e de la sintaxis abstracta, obteniendo como resultado:

```
do {(d1 <- uniform [1 .. 6] | d2 <- uniform [1 .. 6]);
    (d3 <- uniform [1 .. d1] | d4 <- uniform [1 .. d2])} (d3, d4)
```

En la siguiente etapa, el elemento anterior es dado como entrada a la función `desugar`, la cual retornará el código final:

```
join ((\d1 d2 ->
    (\d3 d4 -> (d3, d4) <$> uniform [1 .. d1] <*> uniform [1 .. d2]))
    <$> uniform [1 .. 6] <*> uniform [1 .. 6])
```

No obstante, el plan de ejecución retornado por la etapa de *rearrangement* no es el óptimo para nuestro ejemplo, ya que `s3` no necesita esperar a `s2`, lo que si se está haciendo; lo mismo sucede con `s4` y `s1`.

Lo ideal para nuestro ejemplo sería disponer de dos sub-bloques de cómputo dentro de la notación monádica, de modo que el plan de ejecución óptimo fuera {{s1; s3}|{s2; s4}}. A primera vista, podría parecer que tiene el mismo costo igual a 2. Sin embargo, si modificamos los costos de `s1` y `s4` a 2 unidades cada uno, se observa que el plan de ejecución generado por el algoritmo actual {(s1|s2); (s3|s4)} tendría un costo de 4, mientras que el óptimo {{s1; s3}|{s2; s4}} tendría un costo de 3.

3.2. Mónadas conmutativas

En el caso general, los *statements* dentro de la *do-notation* no pueden reordenarse porque esto alteraría la semántica del programa. Sin embargo, para una subclase particular de mónadas; llamadas *mónadas conmutativas*, este reordenamiento sí es posible sin afectar la semántica del programa. Esta situación se refleja en el ejemplo del lanzamiento de una moneda y un dado presentado en la Sección 1:

<pre>flipCoinRollDice = do c <- uniform [Head, Tail] d <- uniform [1 .. 6] return (c, d)</pre>	$\Leftrightarrow$	<pre>flipCoinRollDice = do d <- uniform [1 .. 6] c <- uniform [Head, Tail] return (c, d)</pre>
--	-------------------	--

Si bien este puede parecer un ejemplo simple, da a entender que la declaración de una expresión monádica mediante la *do-notation*, en el caso de las mónadas conmutativas, puede tener distintas sintaxis que preservan la misma semántica. Esta equivalencia semántica solo es posible cuando se presentan *statements* unitarios independientes o subconjuntos de *statements* que son mutuamente independientes.

Este tipo de mónadas no son consideradas en el algoritmo presentado por Marlow et al., ya que el pipeline de **ApplicativeDo** está diseñado deliberadamente para no reordenar el conjunto de *statements*: la función **rearrange** se limita a agrupar y particionar la secuencia de declaraciones, preservando el orden relativo original. En términos operacionales, esto se manifiesta en que, incluso si el plan de ejecución resultante se representa como un árbol (con secuencias y paralelismos), al “aplanar” dicha representación se recupera nuevamente la lista de *statements* en el mismo orden en que aparecían en el *do* original [7, p. 95].

Esta decisión de diseño es coherente con el caso general: en una mónada arbitraria, reordenar dos declaraciones puede cambiar la semántica del programa cuando existen efectos observables o dependencias sutiles. No obstante, el propio trabajo de Marlow et al. reconoce explícitamente que existe un caso particular donde este impedimento puede levantarse: cuando la mónada satisface una noción de conmutatividad, el intercambio de subcómputos independientes puede preservar la semántica. En ese sentido, el reordenamiento aparece como un “gap” natural del enfoque actual: el algoritmo no lo implementa, pero su pertinencia está motivada por el mismo marco teórico del paper, quedando como una línea plausible de extensión [7, p. 94].

Una gran oportunidad de optimización, por tanto, consiste en incorporar reordenamientos semánticamente válidos (bajo hipótesis de conmutatividad) antes o durante el proceso de *rearrangement*, con el objetivo de ampliar el espacio de planes de ejecución evaluados y, de esta forma, obtener planes con menor costo asociado bajo el modelo del artículo [7]. Esto es particularmente relevante en programas probabilísticos, donde aparecen con frecuencia subcómputos que son independientes en el sentido probabilístico (o cuasi-independientes), pero que no quedan contiguos en el orden sintáctico original y, por ende, no pueden ser separados por cortes simples sin un reordenamiento previo.

En el ejemplo discutido, si se hubiera considerado el intercambio de los *statements* `s2` y `s3` asociados a nuestro ejemplo de los dados, se mantendría la semántica de tal forma:

<pre>do d1 <- uniform [1 .. 6] d2 <- uniform [1 .. 6] d3 <- uniform [1 .. d1] d4 <- uniform [1 .. d2] return (d3, d4)</pre>	$\Leftrightarrow$	<pre>do d1 <- uniform [1 .. 6] d3 <- uniform [1 .. d1] d2 <- uniform [1 .. 6] d4 <- uniform [1 .. d2] return (d3, d4)</pre>
---	-------------------	---

Posteriormente, al ingresar a la etapa de rearrangement, durante la primera iteración de *segments*. Esta habría realizado el corte directamente en `i=2`, debido a que las variables definidas en `{s1, s3}` son `{d1, d3}`, mientras que las variables utilizadas en `{s2, s4}` son únicamente `{d2}`. Al calcular la intersección, esta resulta vacía, por lo que se procede con el corte. Finalmente, el plan de ejecución resultante será `{{s1; s3}|{s2; s4}}`, que corresponde al óptimo esperado. Este plan generaría el siguiente código tras pasar por el proceso de *desugaring*:

```
(\d3 d4 -> (d3, d4))
  <$> (uniform [1 .. 6] >>= (\d1 -> uniform [1 .. d1]))
  <*> (uniform [1 .. 6] >>= (\d2 -> uniform [1 .. d2]))
```

3.2.1. Teoría de distribuciones

La noción de mónada conmutativa resulta particularmente natural en el contexto probabilístico porque, en una formulación estándar, el muestreo de variables aleatorias independientes es conmutativo a nivel semántico: si dos subcómputos no comparten dependencias de datos (no existe relación *def-use* entre sus variables), entonces permutar su orden no altera la distribución conjunta inducida por el programa, sino únicamente el orden sintáctico en que se generan los valores. En otras palabras, el efecto probabilístico asociado a “elegir” un valor para una variable y luego para otra, cuando ambos efectos son independientes, puede interpretarse como un producto de distribuciones que es conmutativo. Esta intuición es precisamente la que se busca capturar cuando se afirma que las mónadas de probabilidad constituyen un caso canónico donde el reordenamiento de *statements* puede preservar la semántica.

Desde un punto de vista más formal, Kock presenta una caracterización axiomática en la cual las mónadas conmutativas pueden entenderse como una teoría general de distribuciones [5]. En este marco, la conmutatividad no se plantea como un “truco” sintáctico, sino como una propiedad algebraica que garantiza que ciertas combinaciones de efectos (por ejemplo, formar distribuciones conjuntas a partir de efectos independientes) son invariantes bajo permutación. Esto aporta una justificación teórica para tratar el caso probabilístico como un candidato privilegiado a extensiones del pipeline de `ApplicativeDo`: si se dispone de criterios para identificar independencia (p. ej., ausencia de dependencias *def-use* entre subcómputos), entonces la conmutatividad permite, en principio, reordenar con seguridad esos subcómputos y exponer paralelismo que el algoritmo actual no alcanza por restringirse a particiones que respetan el orden sintáctico original.

Complementariamente, Jacobs estudia de manera sistemática varias mónadas de probabilidad (incluyendo construcciones clásicas como la mónada de Giry y variantes relacionadas con expectativa), y discute condiciones bajo las cuales estas estructuras admiten nociones de conmutatividad relevantes para modelos probabilísticos [4]. En términos prácticos para esta memoria, estas referencias permiten sostener la idea de que, bajo supuestos razonables (por ejemplo, muestreos independientes y ausencia de efectos observables adicionales), las mónadas de probabilidad se comportan como mónadas conmutativas, por lo que el reordenamiento de *statements* independientes puede considerarse semánticamente válido. En consecuencia, el “salto” desde la teoría a la propuesta técnica es directo: si el compilador puede detectar independencia estructural en un bloque *do*, entonces un paso adicional de reordenamiento, justificado por conmutatividad, puede aumentar el espacio de planes de ejecución posibles y mejorar la paralelización obtenida.

4. Objetivos

Objetivo General

Extender experimentalmente la implementación actual de `ApplicativeDo` en el compilador `GHC`, de modo que pueda considerar reordenamientos semánticamente válidos de *statements* dentro de la *do-notation* cuando se trabaja con mónadas conmutativas, con el fin de mejorar el plan de ejecución generado sin alterar la semántica del programa.

Objetivos Específicos

1. Caracterizar la implementación actual del algoritmo de transformación monádica en el compilador `GHC`, identificando los módulos, estructuras de datos y puntos de extensión relevantes para su modificación.
2. Diseñar e implementar un algoritmo capaz de generar permutaciones válidas de *statements* dentro de una mónada conmutativa, respetando las restricciones **def-use** y preservando la semántica del programa.
3. Integrar las permutaciones válidas generadas al algoritmo propuesto por Marlow et al. [6], de modo de evaluar los planes de ejecución candidatos y seleccionar aquel que minimice el costo según el modelo adoptado.
4. Extender experimentalmente la implementación de `ApplicativeDo` en `GHC` para incorporar el reordenamiento de *statements* dentro del pipeline actual de transformación monádica.
5. Verificar que la extensión propuesta preserve la semántica probabilística del programa original y evaluar su efecto sobre los planes de ejecución obtenidos.

Evaluación

La evaluación del trabajo se realizará a partir de un conjunto de casos de prueba construido sobre dos fuentes complementarias. En primer lugar, se utilizarán como base la *testsuite* oficial de GHC asociados a la extensión `ApplicativeDo` [3], ya que estos constituyen la validación más directa del comportamiento actualmente aceptado por el compilador para la transformación propuesta por Marlow et al. En segundo lugar, se incorporarán casos adicionales diseñados específicamente para esta memoria, orientados a cubrir situaciones donde el nuevo algoritmo de permutación de *statements* dentro de mónadas conmutativas pueda alterar el plan de ejecución sin modificar la semántica probabilística esperada.

La equivalencia semántica entre el programa original y el programa transformado se verificará comparando que ambos induzcan la misma distribución de probabilidad de salida sobre los casos de prueba considerados. En términos prácticos, esto significa comprobar que la transformación preserve los resultados probabilísticos observables del programa, aun cuando se haya modificado el orden sintáctico de los *statements* o la estructura interna del plan de ejecución. De esta manera, el objetivo específico de preservación semántica se evaluará no solo a nivel sintáctico o estructural, sino en función de la distribución probabilística finalmente representada por el programa transformado.

Por otra parte, la noción de optimalidad utilizada en esta memoria se definirá con respecto al sistema de costos simple ya establecido en el artículo base de Marlow et al [7]. En consecuencia, el *baseline* de comparación no estará dado por tiempos reales de ejecución, sino por el costo abstracto asignado a cada plan de ejecución según dicho modelo. Así, para cada permutación válida generada, se calculará el plan de ejecución que produce el algoritmo en la etapa de *rearrangement* y se asociará a dicho plan el costo definido en el paper. Luego, se compararán los costos obtenidos para todas las permutaciones válidas consideradas, y se seleccionará como resultado óptimo el plan de ejecución menos costoso.

5. Solución Propuesta

La solución propuesta consiste en extender experimentalmente la implementación existente de `ApplicativeDo` en el compilador GHC, con el objetivo de que el pipeline actual pueda considerar reordenamientos semánticamente válidos de *statements* dentro de la *do-notation* cuando se trabaja con mónadas conmutativas. En términos generales, la memoria no propone construir un compilador nuevo ni un lenguaje independiente, sino intervenir una transformación ya presente en GHC, tomando como base el algoritmo de *Rearrangement* y *Desugaring* propuesto por Marlow et al. [7]. La idea central es ampliar este pipeline para que no se limite al orden sintáctico original de los *statements*, sino que pueda evaluar permutaciones válidas y seleccionar el plan de ejecución de menor costo según el modelo adoptado, sin alterar la semántica del programa.

Para llevar a cabo esta propuesta, se trabajará sobre un repositorio experimental que ya incorpora un entorno reproducible mediante el uso de un *devcontainer* y un submódulo con el código fuente del compilador **GHC**. Sobre este entorno, será necesario disponer de la cadena de herramientas base del compilador, incluyendo una instalación funcional de **GHC** para bootstrap, *cabal-install* para la gestión y construcción de componentes **Haskell**, y las herramientas *happy* y *alex*, utilizadas por **GHC** para generar analizadores sintácticos y léxicos. Adicionalmente, el proceso de compilación del compilador requiere herramientas estándar del sistema, tales como un compilador de **C** y utilidades de configuración, las cuales se consideran encapsuladas dentro del entorno de desarrollo reproducible preparado para esta memoria.

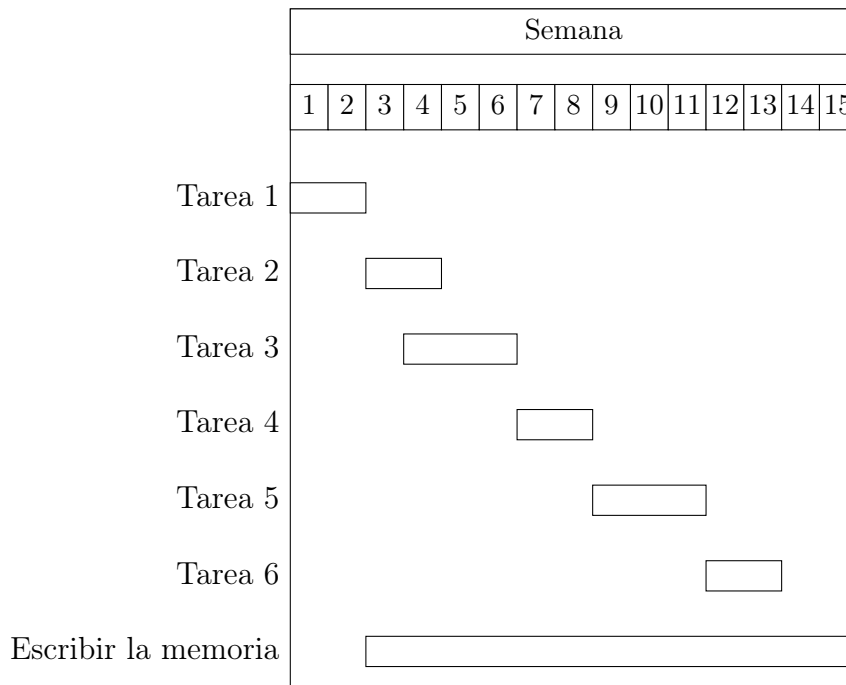
La primera etapa de la solución consistirá en diseñar e implementar un algoritmo capaz de generar todas las permutaciones válidas de *statements* dentro de una mónada conmutativa, respetando las restricciones *def-use* y preservando la semántica del programa. Esta etapa corresponde al componente nuevo de la propuesta y busca ampliar el espacio de configuraciones candidatas que luego serán evaluadas por el algoritmo base de Marlow et al. En particular, el propósito no es reemplazar el pipeline original, sino producir un conjunto de órdenes sintácticos alternativos que sean semánticamente admisibles en el contexto probabilístico y, por tanto, susceptibles de ser ingresados posteriormente al proceso de *Rearrangement* y *Desugaring*.

Una vez generadas las permutaciones válidas, estas se integrarán al algoritmo propuesto por Marlow et al., de modo que cada una produzca un plan de ejecución candidato. Luego, dichos planes se compararán usando el modelo de costos del artículo base, seleccionando el de menor costo. De esta manera, el algoritmo original no se reemplaza, sino que pasa a operar sobre un conjunto más amplio de configuraciones sintácticamente posibles y semánticamente válidas. Sobre esta base, la extensión experimental en **GHC** consistirá en incorporar el reordenamiento de *statements* dentro del pipeline actual de **ApplicativeDo**. Así, la propuesta se sitúa en una modificación controlada del compilador, y no en una transformación externa al mismo. En consecuencia, el objetivo es que el propio compilador pueda considerar órdenes alternativos de evaluación cuando estos sean válidos, y escoger aquel que produzca el plan de ejecución más conveniente según el criterio de costo adoptado.

6. Plan de trabajo

La lista de pasos a seguir para desarrollar la solución propuesta, son los siguientes:

1. **Caracterización en GHC.** Identificación de los módulos, estructuras de datos y puntos de extensión relevantes del algoritmo actualmente implementado.
2. **Diseño del algoritmo de permutaciones válidas.** Definición de los criterios que debe satisfacer una permutación de *statements* dentro de una mónada conmutativa.
3. **Implementación del generador de permutaciones.** Desarrollo del componente encargado de producir las permutaciones válidas que serán evaluadas posteriormente.
4. **Extensión experimental de ApplicativeDo en GHC.** Modificación de la implementación actual del compilador en los puntos de extensión identificados, de modo que el pipeline pueda recibir y procesar órdenes alternativos de *statements* dentro de la transformación monádica.
5. **Integración y selección de planes de ejecución.** Conexión entre la generación de permutaciones válidas, su ingreso al algoritmo de Marlow et al. y la comparación de los planes resultantes para seleccionar el de menor costo según el modelo adoptado.
6. **Evaluación y validación.** Verificación de preservación semántica y análisis de optimalidad mediante el *testsuite* oficial y casos adicionales.



7. Trabajo Adelantado

El trabajo adelantado de esta memoria se ha orientado a reducir el riesgo técnico del proyecto antes de intervenir directamente el algoritmo asociado a `ApplicativeDo` en GHC. En lugar de limitarse a una revisión bibliográfica o a una caracterización puramente conceptual del problema, este avance se concretó en un repositorio experimental funcional [1], reproducible y documentado, preparado específicamente para estudiar el pipeline real del compilador y habilitar modificaciones controladas sobre su implementación. En consecuencia, el trabajo adelantado no solo acredita familiaridad con el problema de investigación, sino que demuestra que ya existe una base técnica concreta sobre la cual desarrollar la propuesta de memoria.

7.1. Infraestructura reproducible y baseline experimental

Un primer avance fundamental fue la consolidación de un entorno reproducible basado en un contenedor `Docker` de desarrollo (*Dev-Container*), con el objetivo de eliminar variaciones del entorno local y asegurar que el proyecto pudiera reconstruirse de manera consistente. Esta decisión fue necesaria porque el trabajo sobre GHC depende de una cadena de herramientas precisas, y pequeñas diferencias de versiones pueden impedir la compilación del compilador o introducir errores difíciles de diagnosticar. En este contexto, se resolvieron incompatibilidades de versiones asociadas a *ghcup*, particularmente problemas de disponibilidad de versiones, y se fijó una combinación funcional de herramientas para el entorno experimental: GHC de bootstrap 9.14.1, `cabal-install` 3.14.2.0, `alex` 3.5.4.0 y `happy` 2.2. Adicionalmente, se incorporaron las dependencias del sistema necesarias para ejecutar `./boot` en GHC, en particular `autoconf`, `automake` y `libtool`, y se añadieron scripts de verificación de la toolchain dentro del directorio `.devcontainer`, de modo que la validación del entorno quede automatizada y no dependa de inspección manual.

En paralelo, se estabilizó la base del compilador utilizada para la experimentación. Inicialmente, se partía desde un commit antiguo de GHC con problemas de reproducibilidad debido a submódulos históricos rotos u obsoletos, lo que hacía riesgosa cualquier intervención posterior. Para superar este problema, se migró a un baseline moderno del submódulo GHC, fijando como commit de referencia `0b36e96cb93db71f201aaa055c4a90b75a8110ef` por su mayor actualidad y no el commit específico asociado a la extensión `ApplicativeDo` (2015) [2]. Sobre esta base se crearon y ajustaron scripts para el alta, inicialización y verificación del submódulo `vendor/ghc`, así como de sus submódulos anidados, y se incorporó automatización mediante un archivo *Makefile* para simplificar el proceso de setup y reproducción. Más importante aún, se tomó la decisión de privilegiar una estrategia de patches sobre el submódulo en lugar de modificar un fork sin trazabilidad, lo que fortalece simultáneamente la reproducibilidad, la mantenibilidad y la trazabilidad de los cambios locales.

7.2. Construcción de GHC y uso controlado del compilador del experimento

Un segundo frente de trabajo consistió en documentar y validar el flujo real de compilación de GHC dentro de `vendor/ghc`. En particular, se dejó establecido el pipeline efectivo de construcción del compilador mediante las etapas `./boot`, `./configure` y `./hadrian/build -j$(nproc)-flavour=devel2`. Esto fue necesario para pasar desde una caracterización teórica

del código fuente a una base experimental verificable, donde el compilador pudiera efectivamente reconstruirse y utilizarse como herramienta de trabajo. Junto con ello, se verificó el uso de los binarios generados en las carpetas *stage1* y *stage2* dentro del submódulo, lo que permitió confirmar que el flujo de build no solo finaliza correctamente, sino que produce artefactos ejecutables aptos para la experimentación.

Sobre esta base, se habilitó explícitamente la compilación de programas del experimento usando **GHC** construido desde el submódulo, en vez del **GHC** global del sistema. Esta decisión es crucial para la validez experimental de la memoria: si las pruebas se ejecutaran con el compilador global, cualquier modificación realizada dentro de *vendor/ghc* dejaría de ser observable en el experimento, comprometiendo la interpretación de resultados. Al usar de forma controlada el compilador reconstruido desde el submódulo, el repositorio experimental garantiza que la observación del comportamiento de **ApplicativeDo** corresponde realmente a la implementación intervenida. En otras palabras, este avance no solo habilita la compilación de **GHC**, sino que asegura que las etapas posteriores de la memoria trabajen sobre una base experimental fiel al código efectivamente modificado.

7.3. Plataforma experimental para observar **ApplicativeDo**

Sobre la infraestructura anterior se desarrolló y refinó una plataforma experimental específica en *experiments/applicative-do/*, destinada a observar de forma directa el comportamiento del compilador frente a distintos patrones de uso de **ApplicativeDo**. El archivo *Main.hs* del experimento fue ampliado hacia un caso más representativo y compacto, incorporando escenarios que ejercitan segmentos applicative puros, dependencias monádicas, **BodyStmt**, patrones estrictos, patrones refutables, bloques anidados y casos que fuerzan el uso de **join**. Esta ampliación fue necesaria para que el experimento no se limitara a ejemplos mínimos, sino que cubriera una variedad de situaciones estructuralmente relevantes para el estudio del algoritmo.

Asimismo, se implementó *run.sh* para compilar y ejecutar el experimento de forma reproducible, con autodetección del compilador **GHC** construido desde el submódulo y soporte para la variable *GHC_BIN*. A esto se sumó la incorporación de flags orientados a la observabilidad del pipeline del compilador: *DUMP_DS=1* para obtener el dump de desugaring, *DUMP_PIPELINE=1* para generar dumps del renamer, typechecker y desugar (*-ddump-rn*, *-ddump-tc*, *-ddump-ds*), *DUMP_SIMPL=1* para obtener el Core simplificado, y *EXTRA_GHC_FLAGS="..."* para extender la observación con opciones adicionales. También se agregó un manifiesto de dumps generados en *build/dump-manifest.txt*. El valor de esta plataforma es doble: por una parte, permite explorar de manera controlada el proceso de **Rearrangement** y **Desugar** asociado a **ApplicativeDo**; por otra, constituye una base práctica para contrastar futuras modificaciones del algoritmo contra ejemplos concretos y artefactos verificables del compilador.

7.4. Documentación técnica, trazabilidad e impacto en la viabilidad

Un último bloque de trabajo adelantado fue la consolidación de documentación técnica específica, orientada no solo a describir el repositorio, sino a capturar conocimiento operativo y arquitectónico que de otro modo quedaría disperso o implícito. En particular, se actualizaron *README.md*, *docs/toolchain.md*, *docs/architecture.md* y *experiments/applicative-do/README.md*. Dentro de este conjunto, *docs/architecture.md* tiene un valor especialmente alto para la memoria, pues deja trazado un mapa exhaustivo de **ApplicativeDo** en GHC actual: se describe el recorrido Parser -> Renamer -> Typechecker -> Zonk -> Desugar, las estructuras **ApplicativeStmt** y **ApplicativeArg**, funciones críticas como **rearrangeForApplicativeDo**, **segments**, **mkApplicativeStmt**, **tcApplicativeStmts**, **dsDo**, entre otras, así como los puntos de extensión más relevantes para experimentación. Esto transforma un conocimiento que normalmente requeriría una exploración larga y costosa del código fuente en un activo reutilizable y verificable.

En conjunto, estos avances permiten afirmar que el proyecto ya dispone de una base experimental real sobre la cual construir la investigación propuesta. Se cuenta con un entorno reproducible y validado, con una estrategia clara de mantenimiento sobre GHC, con un baseline moderno y verificable, con una ruta documentada de compilación del compilador, con una plataforma experimental que permite observar **ApplicativeDo** en funcionamiento y con documentación técnica suficiente para localizar los puntos donde la memoria deberá intervenir. Desde el punto de vista metodológico, esto mejora la reproducibilidad, fortalece la trazabilidad, incrementa la capacidad experimental y reduce de manera importante el riesgo técnico del proyecto. En consecuencia, la memoria puede concentrarse en su núcleo de investigación, es decir, la generación de permutaciones válidas de statements, su integración al algoritmo base y el estudio de su efecto sobre los planes de ejecución, todo esto sobre una base robusta y verificable.

Referencias

- [1] Esteban Romero Berríos. Repositorio experimental para el estudio de applicative-do y reordenamiento de statements en ghc, 2026. GitHub repository. URL: <https://github.com/estebanrb22/trabajo-de-titulo-applicative-do-reordering>.
- [2] GHC Contributors. Commit 8ecf6d8f7dfce9e5b1844cd196f83f00f3b6b879, 2015. GitHub commit. URL: <https://github.com/ghc/ghc/commit/8ecf6d8f7dfce9e5b1844cd196f83f00f3b6b879>.
- [3] GHC Contributors. Testsuite de la extensión applicativo en ghc, 2026. Directorio `testsuite/tests/ado` del repositorio oficial de GHC. URL: <https://github.com/ghc/ghc/tree/master/testsuite/tests/ado>.
- [4] Bart Jacobs. From probability monads to commutative effectuses. *Journal of Logical and Algebraic Methods in Programming*, 94:200–237, 2018. doi:10.1016/j.jlamp.2016.11.006.
- [5] Anders Kock. Commutative monads as a theory of distributions. *Theory and Applications of Categories*, 26(4):97–131, 2012. URL: <https://www.tac.mta.ca/tac/volumes/26/4/26-04.pdf>.
- [6] Simon Marlow. `ado`, 2015. GitHub repository. URL: <https://github.com/simonmar/ado>.
- [7] Simon Marlow, Simon Peyton Jones, Edward Kmett, and Andrey Mokhov. Desugaring Haskell’s `do`-notation into applicative operations. In *Proceedings of the 9th International Symposium on Haskell*, pages 92–104. ACM, 2016. doi:10.1145/2976002.2976007.
- [8] Conor McBride and Ross Paterson. Applicative programming with effects. *Journal of Functional Programming*, 18(1):1–13, 2008. doi:10.1017/S0956796807006326.
- [9] Justin Pombrio. *Resugaring: Lifting Evaluation Sequences through Syntactic Sugar*. PhD thesis, Brown University, 2018. URL: <https://cs.brown.edu/research/pubs/theses/phd/2018/pombrio.justin.pdf>.
- [10] Adam Ścibior, Zoubin Ghahramani, and Andrew D. Gordon. Practical probabilistic programming with monads. In *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell*, pages 165–176. ACM, 2015. doi:10.1145/2804302.2804317.
- [11] Philip Wadler. Monads for functional programming. In *Advanced Functional Programming*, volume 925 of *Lecture Notes in Computer Science*, pages 24–52. Springer, 1995. doi:10.1007/3-540-59451-5_2.
- [12] Brent Yorgey. The typeclassopedia. *The Monad.Reader*, 13:17–68, 2009. URL: <https://wiki.haskell.org/wikiupload/e/e9/Typeclassopedia.pdf>.